CS 175: Project in AI — Spring 2026

Project Proposal AI2-THOR

Title: Explicit vs. Implicit Memory and Semantic Priors in Object Goal Navigation: A Comparative Study in AI2-THOR and ProcTHOR

Team Members: Charles Gao, Emily Sun, Junyuan Zhang, Tong Zhao

Group Number: 66

Note: We are a 4-member team. Per the course syllabus, we have requested approval from Prof. Kask.

I. Main Idea

The goal of this project is to build an embodied agent that can find a specified household object inside a 3D house it has never seen before. Given a target object such as “microwave,” “apple,” or “television,” and a random starting position in a simulated house, the agent must navigate through rooms using first-person RGB-D observations and call Stop when the target is in view. From the user’s perspective, the system takes one text input, such as “find the microwave,” and then explores the home on its own until it either finds the object or runs out of steps.

Two capabilities matter most for this task. The first is memory of the environment. After walking through a kitchen and a bedroom, the agent should not keep returning to the same places while searching for a fridge. The second is common sense about where objects are likely to appear. When asked to find a microwave, the agent should prefer a kitchen over a bathroom before it has seen the target.

Different approaches encode these abilities differently. An end-to-end neural policy stores them implicitly inside network weights and recurrent hidden states. A classical mapping-and-planning pipeline stores memory explicitly as a 2D semantic map. An LLM-guided agent can combine explicit map memory with explicit commonsense reasoning by asking a language model where the target is likely to be. Our project compares these three paradigms on AI2-THOR and ProcTHOR to answer one question: does explicit structure, including map-based memory and LLM-based priors, improve object goal navigation over end-to-end black-box learning?

The techniques studied here are relevant to home service robots, assistive technology for users with visual impairments, and warehouse or hospital logistics robots.

II. Input / Output

At each timestep, the agent receives a first-person RGB image, a depth image, and a compass reading describing orientation and displacement relative to the start of the episode. The target object category is provided at the start, such as AlarmClock, Apple, Bowl, Laptop, Mug, Television, or Vase. For the LLM method, the target is also provided as a text string.

At each timestep, the agent outputs one of six discrete actions: MoveAhead, RotateLeft, RotateRight, LookUp, LookDown, or Stop. Stop is how the agent declares it has found the target. An episode ends when the agent calls Stop or reaches the maximum number of timesteps.

Methods 2 and 3 also maintain a 2D top-down semantic grid map of the scene seen so far. This map records explored areas, traversable cells, and detected object classes. It serves both as explicit memory and as a visualization for qualitative analysis.

The reward structure is simple. The agent receives a positive terminal reward for calling Stop when the target is visible and close enough, a negative reward for calling Stop incorrectly, and a small step penalty to encourage efficient exploration. We may also include an optional shaping reward based on progress toward the nearest target instance. Success follows the standard ObjectNav convention: the target must be visible in the final RGB frame and close enough to the agent.

III. Basic Approach

We implement and compare three methods that differ along two axes. Memory can be implicit, stored in a network hidden state, or explicit, stored as a 2D semantic map. Semantic priors can be implicit, learned

from training data, or explicit, supplied through a language model.

All implementations will use PyTorch. RL training will be built on AllenAct, which provides ObjectNav infrastructure for AI2-THOR and ProcTHOR.

Method 1: End-to-End RL (Baseline)

A ResNet-18 encodes each RGB-D frame. The encoding is concatenated with the target-category embedding and compass reading, passed through a GRU for short-term memory, and decoded into policy and value heads. The network is trained with PPO on ProcTHOR houses. Memory exists only inside the GRU hidden state, and semantic priors exist only in the trained weights. This is the standard black-box embodied RL baseline.

Method 2: Semantic Mapping + Classical Planner (Explicit Memory)

For perception, a pretrained Detic object detector runs on each RGB frame and produces object labels. Using the depth image and the agent's pose, these labels are projected into world coordinates and accumulated into a 2D top-down semantic grid map. Each cell stores which object classes have been seen there and whether the cell is traversable.

For planning, the agent follows a simple rule. If the target object appears on the map, it runs A* to navigate to it and calls Stop when close enough. Otherwise, it performs frontier-based exploration by finding boundaries between explored and unexplored space and moving toward the most promising frontier. No LLM is involved. This is a SemExp-style classical pipeline. Memory is now fully explicit: the agent remembers where it has been. However, semantic priors are still limited to exploration heuristics.

Method 3: Semantic Map + LLM-Guided Exploration (Explicit Memory + Explicit Priors)

Method 3 uses the same mapping module as Method 2. The difference is how the next movement is chosen. Every N steps, such as every 20 steps, the current semantic map is converted into a short natural-language summary. For example: "You are looking for a microwave. You have explored three regions. Region A contains a fridge, stove, and kitchen counter. Region B contains a bed and nightstand. Region C contains a toilet and sink. Unexplored areas remain to the north and east. Which direction should you go next, and why?"

The LLM replies with a reasoning trace and a chosen target region or unexplored direction. A low-level A*-based planner executes the movement. We will use Claude 3.5 Haiku as the primary model for cost reasons and GPT-4o as an upper-bound reference if time permits. We will compare zero-shot direct prompting with ReAct-style reasoning and action prompting.

This method combines the map-based memory of Method 2 with the commonsense priors of a language model. It captures the behavior that motivates this project: the agent remembers where it has been and knows that microwaves usually belong in kitchens.

Division of Labor (4 Members)

Charles Gao will implement and train Method 1, build parts of the shared evaluation pipeline, and visualize training curves. Emily Sun will build the semantic mapping module, including detector integration, depth projection, and the 2D grid data structure shared by Methods 2 and 3. Junyuan Zhang will complete Method 2, including frontier exploration, A* planning, and shared motion primitives. Tong Zhao will complete Method 3, including map-to-text serialization, prompt design, ReAct loop, API-cost

tracking, and analysis of LLM reasoning. All four members will contribute to experiment design, the final report, and the presentation.

IV. Evaluation Plan

All three methods will be evaluated on identical test sets with identical seeds. The test protocol has two levels of generalization. First, we evaluate on held-out ProcTHOR houses that are never seen during training. This measures generalization to new layouts within the ProcTHOR distribution. Second, we evaluate on RoboTHOR validation scenes. This measures transfer to a different scene distribution, where explicit structure may be especially useful.

The primary quantitative metrics are Success Rate, SPL, and SoftSPL. Success Rate measures the percentage of episodes that end in a correct Stop. SPL measures success weighted by path efficiency. SoftSPL gives partial credit for ending near the target even without a perfect Stop. Secondary metrics include sample efficiency for Method 1, wall-clock running time per episode, generalization gap, API cost per solved episode for Method 3, and per-category Success Rate.

We will also perform qualitative analysis. For Methods 2 and 3, we will visualize the top-down semantic map with the agent's trajectory and detected objects. This map shows what the agent remembers and how it uses memory. For Method 3, we will log the LLM's reasoning at each decision step and check whether it makes coherent choices, such as choosing a kitchen-like region when searching for a microwave.

Failed episodes will be categorized into exploration failure, decision failure, recognition failure, and false-positive Stop. We expect Methods 2 and 3 to have fewer exploration failures because they explicitly remember searched areas. We will also collect episode videos showing successes and instructive failures.

V. Expectations, Backup Plan, and Milestones

We expect the random baseline to perform poorly. Method 1 should improve with PPO training but may show weaker transfer to RoboTHOR. Method 2 may have similar or slightly lower in-distribution performance but a smaller out-of-distribution gap because explicit memory transfers across scenes. Method 3 should have the strongest transfer when object-location commonsense matters, especially for targets such as microwave, bed, television, or toilet.

If time permits, we will test zero-shot object categories that were not encountered during training, such as toaster, pillow, or book. We expect Method 3's LLM priors to generalize better than a trained RL policy. We may also test how sensitive Methods 2 and 3 are to detector quality by replacing Detic with a smaller detector.

If AI2-THOR or ProcTHOR cannot run reliably on our available hardware by the end of Week 2, we will fall back to a simpler iTHOR single-room setup using the 120 iTHOR scenes. The three-method comparison and the core question of implicit versus explicit memory and priors will remain the same.

Week 1 to 2: set up AI2-THOR, ProcTHOR, AllenAct, PyTorch, Detic, the GitHub repository, and the random-agent baseline. Week 3 to 4: start Method 1 training, build the first semantic map, prototype the planner and LLM prompt, and develop the shared evaluation pipeline. Week 5 to 6: complete Method 2 and Method 3, integrate map-to-text-to-LLM-to-action, and run initial evaluations on a development set. Later weeks focus on full evaluation, comparison plots, semantic map visualizations, LLM trace figures, the final report, demo video, and presentation.

References

Kolve, E., et al. (2017). AI2-THOR: An Interactive 3D Environment for Visual AI. arXiv:1712.05474.

Deitke, M., et al. (2020). RoboTHOR: An Open Simulation-to-Real Embodied AI Platform. CVPR.

Deitke, M., et al. (2022). ProcTHOR: Large-Scale Embodied AI Using Procedural Generation. NeurIPS.

Weihs, L., et al. (2021). AllenAct: A Framework for Embodied AI Research. arXiv:2008.12760.

Chaplot, D. S., et al. (2020). Object Goal Navigation using Goal-Oriented Semantic Exploration (SemExp). NeurIPS.

Chaplot, D. S., et al. (2020). Learning to Explore using Active Neural SLAM. ICLR.

Zhou, X., et al. (2022). Detecting Twenty-thousand Classes using Image-level Supervision (Detic). ECCV.

Schulman, J., et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.

Yao, S., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR.

Song, C. H., et al. (2023). LLM-Planner: Few-Shot Grounded Planning for Embodied Agents with Large Language Models. ICCV.

Anderson, P., et al. (2018). On Evaluation of Embodied Navigation Agents. arXiv:1807.06757. (SPL metric.)

Henderson, P., et al. (2018). Deep Reinforcement Learning That Matters. AAAI. (RL evaluation methodology.)

AI2-THOR documentation: https://ai2thor.allenai.org/

ProcTHOR repository: https://github.com/allenai/procthor